Assignment 3 Report: End-to-End Hugging Face Model Training & Docker Deployment

Name: Sushantak Parashar Jha

Roll No: M25CSA035

Course: ML-DL-Ops

Institute: IIT Jodhpur

---

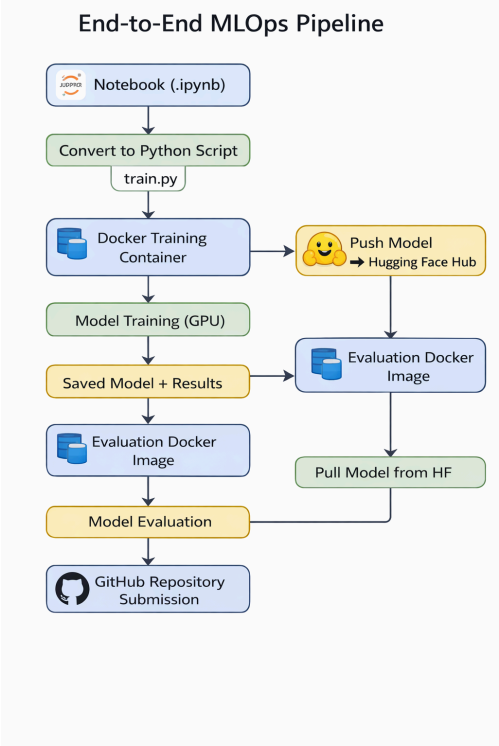
1. Objective

The objective of this assignment was to design and implement a complete end-to-end Machine Learning Operations (MLOps) workflow. The pipeline includes model training, evaluation, containerization using Docker, deployment using the Hugging Face Hub, and version control using GitHub to ensure reproducibility and portability of machine learning experiments.

---

2. Overall Workflow

The complete workflow followed in this assignment is outlined below in the figure:



---

3. Environment Setup Using Docker

Docker was used to create an isolated and reproducible environment independent of the host system.

Steps Performed:

- Created Dockerfile
- Installed dependencies
- Configured working directory
- Enabled GPU support

Docker Build Command:

```
docker build -t hf-train:v1 .
```

Docker Run Command:

```
docker run -it --rm \
--gpus all \
--shm-size=8g \
-v $(pwd):/workspace \
hf-train:v1
```

---

4. Notebook Conversion to Python Script

The notebook was converted into the Python script.

Steps:

1. Downloaded notebook
2. Converted into Python script
3. Removed notebook artifacts
4. Organized execution flow

Command used:

```
jupyter nbconvert --to python notebook.ipynb
```

Final script used: train.py

---

5. Model Selection

A pre-trained transformer model (DistilBERT) from Hugging Face was selected.

Reasons for Selection:

- Lightweight architecture
- Faster training compared to standard BERT
- Good performance for text classification tasks
- Efficient fine-tuning capability

Note: The tokenizer and model were loaded directly from Hugging Face.

---

6. Model Training Using Hugging Face Trainer API

The model was fine-tuned using the Hugging Face Trainer API.

Training Configuration:

Parameter	Value
Epochs	3
Batch Size (Training)	10
Batch Size (Evaluation)	16
Learning Rate	2.00E-05
Optimizer	AdamW
Evaluation Strategy	Epoch-wise Evaluation
Device	GPU (CUDA enabled)

Training included:

- Dataset preprocessing
- Tokenization
- Trainer configuration
- GPU-accelerated training

---

7. Model Evaluation

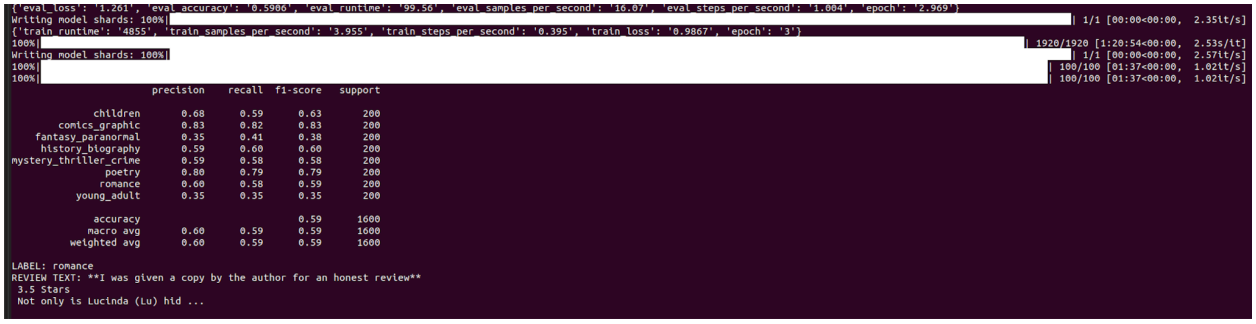
After training, the model was evaluated on the validation/test dataset.

Metrics Used:

- Accuracy
- F1 Score
- Loss

Output:

Class	Precision	Recall	F1-Score	Support
children	0.68	0.59	0.63	200
comics_graphic	0.83	0.82	0.83	200
fantasy_paranormal	0.35	0.41	0.38	200
history_biography	0.59	0.6	0.6	200
mystery_thriller_crime	0.59	0.58	0.58	200
poetry	0.8	0.79	0.79	200
romance	0.6	0.58	0.59	200
young_adult	0.35	0.35	0.35	200
accuracy			0.59	1600
macro avg	0.6	0.59	0.59	1600
weighted avg	0.6	0.59	0.59	1600



The results confirmed the successful fine-tuning of the pre-trained model.

---

8. Saving and Uploading Model to Hugging Face

A Hugging Face account was created, and an access token was generated to push the model to the Hub.

Login Command:

python
from huggingface_hub import login
login()

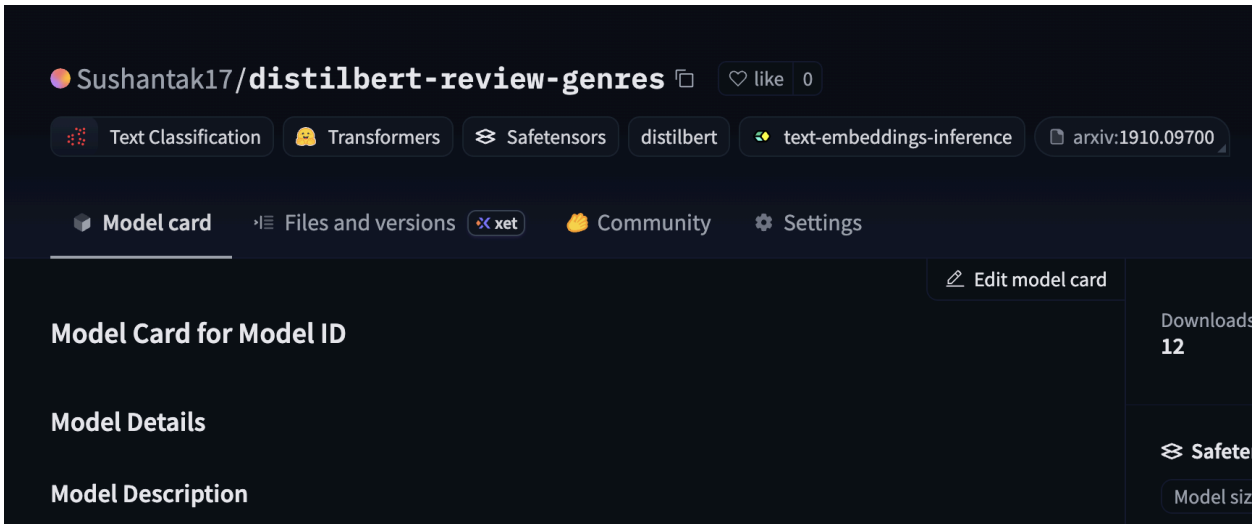
Model Upload:

python
model.push_to_hub("Sushantak17/distilbert-review-genres")

The following artifacts were uploaded:

- Model weights
- Tokenizer
- Configuration files

Hugging Face Model Link: Link



---

9. Re-evaluation from Hugging Face Repository

A separate evaluation Docker container was created, which automatically downloaded the model from Hugging Face and executed the evaluation.

Evaluation Image Build:

Bash
docker build -t hf-eval:v1 -f Dockerfile.eval .

Run Evaluation:

Bash
docker run -it --rm --gpus all hf-eval:v1

Output:

Model loaded successfully from Hugging Face

Observation: The evaluation results were consistent with local training results, confirming correct deployment.

---

10. Final Evaluation Docker Image

A lightweight production Docker image was created specifically for inference purposes.

Purpose:

- Separate training and inference environments
- Establish a reproducible evaluation setup
- Ensure production-ready deployment

---

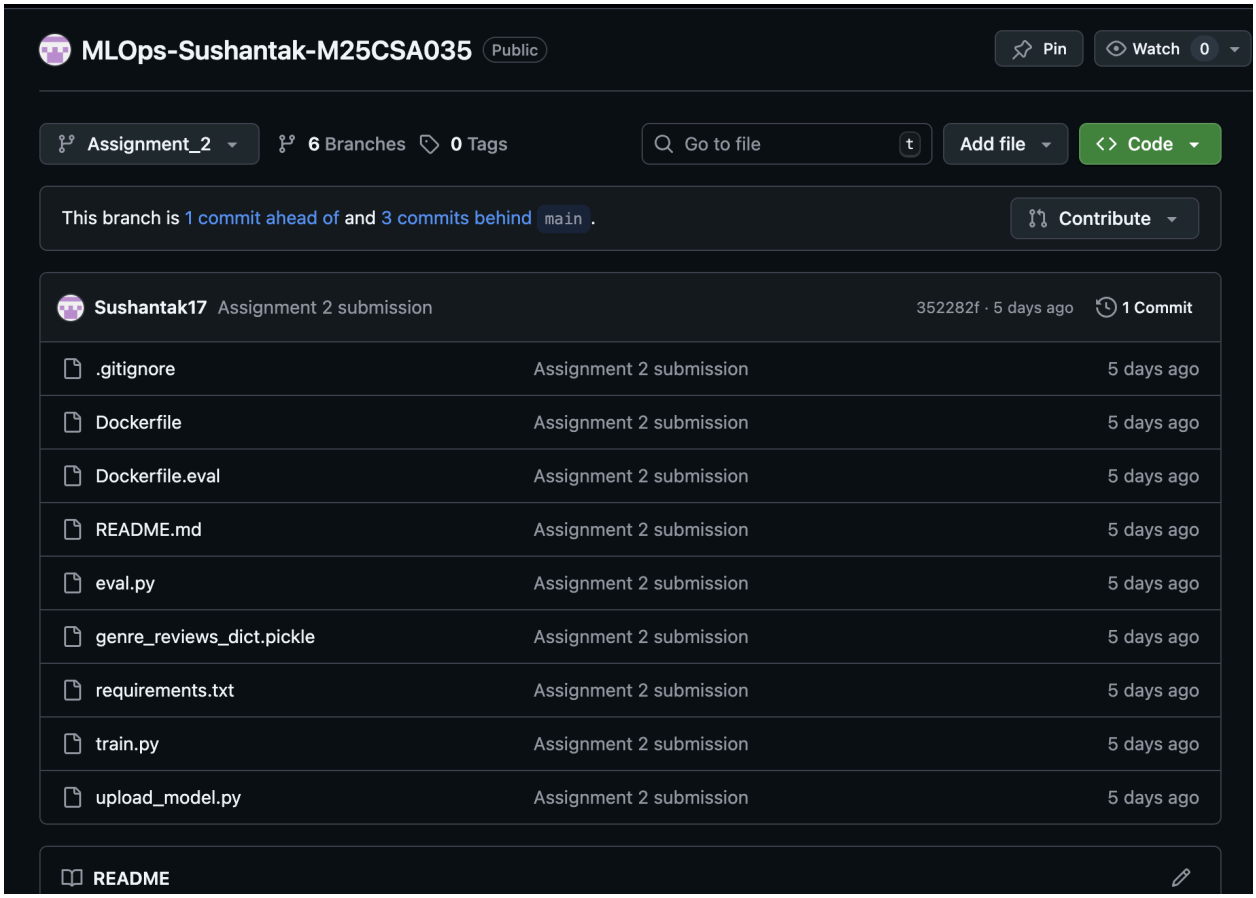
11. GitHub Repository

All project files were version-controlled and pushed to GitHub.

Repository Includes:

- train.py
- eval.py
- Dockerfile
- Dockerfile.eval
- requirements.txt
- README.md

GitHub Repository Link: Link



---

12. Challenges Faced

During implementation, several challenges were encountered:

- Dependency conflicts inside Docker containers
- GPU configuration issues
- Missing libraries during container execution
- Docker image size management

These issues were resolved through iterative debugging and environment configuration.

---

13. Key Learnings

This assignment provided practical exposure to:

- End-to-end MLOps workflows
- Docker containerization
- Hugging Face model deployment
- Reproducible ML experiments

- Separation of training and inference environments

---

14. Conclusion

The assignment successfully demonstrated a complete machine learning lifecycle, starting from experimentation to deployment. Using Docker ensured reproducibility, Hugging Face enabled easy model sharing, and GitHub provided structured version control, collectively forming a production-ready MLOps pipeline.